

APPROACH

GitHub Copilot - AI Augmented Quality Engineer

<https://myskillspring.cognizant.com/learner/assemblyDetails/a72d42d9-5cdf-473f-8b99-a1b43893130c>

Smart university

Parabank

Start assessment > Pre-requisite course > Tekstac > VDI > VS code > 500733_cgcp > Company mail ID
> Authentication

500733_cgcp

Use Claude sonnet 5

Give it as single prompt

It takes 30 mins to run (should give allow)

Defect report generated

Parabank Question

Description

UI AUTOMATION — PARABANK REGISTRATION, OPEN NEW ACCOUNT & FUND TRANSFER

Supports: Any automation tool (Selenium, Playwright, Cypress) in any language (Java, JavaScript, TypeScript, Python, C#, .NET)

OVERVIEW

This assignment requires you to analyze requirements, generate test cases, automate scenarios, execute tests, identify defects, and submit a structured defect report for the Registration, Open New Account, and Fund Transfer modules of the ParaBank application.

IMPORTANT: HOW THIS ASSIGNMENT IS EVALUATED

This assignment does NOT just evaluate the quality of your final code or test cases. It evaluates HOW you collaborated with AI to produce them.

The evaluation measures two things separately:

1. What you analyzed and discovered — and whether you shared it with AI (Application Analysis, Acceptance Criteria, Validation Rules)
2. How you structured and directed your AI prompts (Research prompts, generation prompts, code prompts, refinement prompts)

IMPORTANT GUIDELINES

Do not change the provided workspace structure at any cost. You may implement your framework on top of the existing workspace, but the structure must remain unchanged.

Execution Method: Use the integrated terminal in VS Code for all test execution.

Avoid redundant files and folders and keep the workspace clean and organized.

Create a single defect report named `defect_report.json` consolidating all identified defects. This file must be present in your workspace after execution.

Follow industry-standard best practices for framework design, naming conventions, and code organization.

APPLICATION UNDER TEST

Registration: <http://webapps.tekstac.com:9090/tekbank/register.htm>

The application URL has also been provided in the **user stories** available in your project workspace.

Open New Account and Fund Transfer: Access these modules by navigating the application after login. No separate entry URLs are required.

SCOPE

Module 1 — Registration: 11 fields, US-01 to US-07
Module 2 — Open New Account: Account Type and Source Account, US-08 to US-10
Module 3 — Fund Transfer: Amount and Account Selection, US-11 to US-13

Note: Please refer to the use cases provided in the user story, as outlined in the project template.

HOW TO APPROACH THIS EXERCISE

Each step below corresponds to criteria that are scored in your evaluation.
Skipping a step means scoring 0 on the corresponding criteria.

Step 1 — Inspect All Three Modules

Open each form. Identify HTML element IDs, field types, and behaviors.
Submit empty forms and invalid values to observe how the application responds.

Step 2 — Define Acceptance Criteria

Create acceptance criteria for user stories US-01 through US-13.
AI assistance is allowed, but your prompts must show you are driving the definition.
Save your findings in **Application Analysis & Requirements.txt**.

Step 3 — Research Validation Rules

Identify and document all field-level validation rules for all three modules.
Add these rules to Application Analysis & Requirements.txt before proceeding.

Step 4 — Generate Comprehensive Test Cases With Full Context (.csv)

Use the acceptance criteria, element IDs, and validation rules gathered in Steps 1 to 3 as context in your prompt. Include all user story links and specify the required test distribution before generating.

Step 5 — Generate Automation Code

Use AI to implement browser setup, page objects, test methods, and helper utilities.
Reference the element IDs and validation rules from your analysis in each prompt.

Step 6 — Refine Outputs

Run the generated tests and submit follow-up prompts to fix failures, correct assertions, and improve coverage based on what you observe during execution.

Step 7 — Document Defects

Execute tests and document defects based on actual observed behavior.
Each defect must include what was entered, what happened, and what was expected.
Save all defects in a single file named **defect_report.json** in your workspace.

VALIDATION RULES

Registration Module

Name fields and City	Letters only — numbers and special characters must be rejected
State	Exactly 2 alphabetic characters
ZIP Code	Exactly 5 numeric digits
Phone	Exactly 10 numeric digits
SSN	Exactly 9 numeric digits
All 11 fields	Mandatory — empty submission must show a validation error

Open New Account Module

Account Type	Must be selected (CHECKING or SAVINGS)
Source Account	Must be selected from dynamically loaded dropdown
\$100.00 label	This is a static display label — not a field to test or interact with

Fund Transfer Module

Amount	Mandatory, must be numeric, must be positive (greater than zero)
From Account	Must be selected from dynamically loaded dropdown
To Account	Must be selected from dynamically loaded dropdown

TASKS

Task 1 — Test Case Generation

Generate at least 60 detailed test cases in CSV format covering all 13 user stories across all 3 modules.

Required Columns:

TC_ID, Module, User_Story_ID, Test_Scenario, Test_Type, Priority, Precondition, Test_Steps, Test_Data, Expected_Result, Actual_Result, Status

Priority Values: Critical, High, Medium, Low

Distribution: 60-70% of cases must be negative/validation tests (Critical or High priority). Positive/success scenarios should be Low priority. Boundary tests should be Medium.

Task 2 — Automation Implementation

Create a complete automation solution covering all three modules.

Implement at least 20 automated test methods with assertions.

Prioritize critical and high-risk validations over happy-path scenarios.

Task 3 — Execution and Bug Detection

Execute automation. Provide execution evidence. Save all defects in a single consolidated file named defect_report.json. Detect and document at least 8 validation defects across all modules. Defects must be from actual test execution — not AI-predicted.

DELIVERABLES

1. Application Analysis & Requirements.txt — your documented analysis of all three modules including element IDs, acceptance criteria, and validation rules
2. Test case document — 60+ test cases covering all three modules in a csv file format
3. Automation codebase — page objects for Registration, Open New Account, Fund Transfer
4. defect_report.json — one consolidated defect report in JSON format capturing all defects from actual test execution

MINIMUM PASSING EXPECTATIONS

Application Analysis & Requirements.txt present in workspace with documented findings
60+ test cases covering all 13 user stories
Risk-based coverage with 60-70% negative test scenarios
At least 20 automated tests with meaningful assertions
Page objects for Registration, Open Account, and Transfer pages

defect_report.json present in workspace with at least 8 documented defects,
each including expected result, actual result, severity, and priority

=====

APPLICATION ANALYSIS & REQUIREMENTS DOCUMENT

Target Application: ParaBank Demo (Tekstac Hosted)

Base Registration URL: <http://webapps.tekstac.com:9090/tekbank/register.htm>

=====

1. MODULE & FIELD ELEMENT ID MAP

Module 1: Registration Page (<http://webapps.tekstac.com:9090/tekbank/register.htm>)

- First Name : id="customer.firstName"
- Last Name : id="customer.lastName"
- Address (Street) : id="customer.address.street"
- City : id="customer.address.city"
- State : id="customer.address.state"
- Zip Code : id="customer.address.zipCode"
- Phone Number : id="customer.phoneNumber"
- SSN : id="customer.ssn"
- Username : id="customer.username"
- Password : id="customer.password"
- Confirm Password : id="repeatedPassword"
- Register Button : input[type="submit"][value="Register"]

Module 2: Open New Account Page (Post-login Navigation)

- Account Type : select id="type" (Options: CHECKING, SAVINGS)
- Source Account : select id="fromAccountId" (Dynamic Loading Dropdown)
- \$100.00 Label : Static text element (Not interactive)
- Open Account Button : input[type="submit"][value="Open New Account"]

Module 3: Fund Transfer Page (Post-login Navigation)

- Transfer Amount : id="amount"
- From Account : select id="fromAccountId" (Dynamic Loading Dropdown)
- To Account : select id="toAccountId" (Dynamic Loading Dropdown)
- Transfer Button : input[type="submit"][value="Transfer"]

2. VALIDATION RULES MATRIX

Field / Module	Rule Description & Constraints
First / Last / City	Letters only. Reject numbers and special characters.
State	Exactly 2 alphabetic characters (e.g., "CA", "NY").
Zip Code	Exactly 5 numeric digits (e.g., "90210").
Phone Number	Exactly 10 numeric digits (e.g., "9876543210").
SSN	Exactly 9 numeric digits (e.g., "123456789").
All 11 Reg Fields	Mandatory. Blank submissions must trigger validation errors.
Account Type	Mandatory selection (CHECKING or SAVINGS).
Source/From Account	Mandatory selection from dynamically loaded options.
To Account	Mandatory selection from dynamically loaded options.
Transfer Amount	Mandatory, numeric, positive value (> 0.00).

3. ACCEPTANCE CRITERIA (US-01 through US-13)

- US-01 [Reg - Names]: First Name and Last Name accept alphabetic characters only; show errors for digits/symbols.
- US-02 [Reg - Address]: Street accepts standard address format; City accepts letters only.
- US-03 [Reg - State]: State accepts exactly 2 uppercase/lowercase alphabetic characters.
- US-04 [Reg - Zip]: Zip Code accepts exactly 5 numeric digits.
- US-05 [Reg - Phone & SSN]: Phone requires 10 numeric digits; SSN requires 9 numeric digits.

US-06 [Reg - Credentials]: Username, Password, and Confirm Password must match and meet security length rules.

US-07 [Reg - Mandatory]: Form submission without required fields blocks submission with clear field errors.

US-08 [Open Acc - Type]: User can select valid Account Type (CHECKING/SAVINGS).

US-09 [Open Acc - Source]: Source Account dropdown populates dynamically with existing user accounts.

US-10 [Open Acc - Submission]: Submitting valid account parameters opens new account and shows confirmation ID.

US-11 [Transfer - Amount]: Amount field accepts positive numeric values (>0); blocks negative/zero/non-numeric.

US-12 [Transfer - Selection]: From Account and To Account dropdowns populate dynamically with valid accounts.

US-13 [Transfer - Execution]: Valid transfer updates account balances and displays transaction success page.

Userstories txt:

SCOPE

Module 1 — Registration: 11 fields, US-01 to US-07

Module 2 — Open New Account: Account Type and Source Account, US-08 to US-10

Module 3 — Fund Transfer: Amount and Account Selection, US-11 to US-13

Note: Please refer to the use cases provided in the user story, as outlined in the project template.

HOW TO APPROACH THIS EXERCISE

Each step below corresponds to criteria that are scored in your evaluation.

Skipping a step means scoring 0 on the corresponding criteria.

Step 1 — Inspect All Three Modules

Open each form. Identify HTML element IDs, field types, and behaviors.

Submit empty forms and invalid values to observe how the application responds.

Step 2 — Define Acceptance Criteria

Create acceptance criteria for user stories US-01 through US-13.

AI assistance is allowed, but your prompts must show you are driving the definition.

Save your findings in `Application Analysis & Requirements.txt`.

Step 3 — Research Validation Rules

Identify and document all field-level validation rules for all three modules.

Add these rules to `Application Analysis & Requirements.txt` before proceeding.

Step 4 — Generate Comprehensive Test Cases With Full Context (.csv)

Use the acceptance criteria, element IDs, and validation rules gathered in Steps 1 to 3 as context in your prompt. Include all user story links and specify the required test distribution before generating.

Step 5 — Generate Automation Code

Use AI to implement browser setup, page objects, test methods, and helper utilities.

Reference the element IDs and validation rules from your analysis in each prompt.

Step 6 — Refine Outputs

Run the generated tests and submit follow-up prompts to fix failures, correct assertions, and improve coverage based on what you observe during execution.

Step 7 — Document Defects

Execute tests and document defects based on actual observed behavior.

Each defect must include what was entered, what happened, and what was expected.

Save all defects in a single file named `defect_report.json` in your workspace.

VALIDATION RULES

Registration Module

Name fields and City Letters only — numbers and special characters must be rejected

State Exactly 2 alphabetic characters

ZIP Code Exactly 5 numeric digits

Phone Exactly 10 numeric digits

SSN Exactly 9 numeric digits

All 11 fields Mandatory — empty submission must show a validation error

Open New Account Module

Account Type Must be selected (CHECKING or SAVINGS)

Source Account Must be selected from dynamically loaded dropdown

\$100.00 label This is a static display label — not a field to test or interact with

Fund Transfer Module

Amount Mandatory, must be numeric, must be positive (greater than zero)

From Account Must be selected from dynamically loaded dropdown

To Account Must be selected from dynamically loaded dropdown

TASKS

Task 1 — Test Case Generation

Generate at least 60 detailed test cases in CSV format covering all 13 user stories across all 3 modules.

Required Columns:

TC_ID, Module, User_Story_ID, Test_Scenario, Test_Type, Priority, Precondition, Test_Steps, Test_Data, Expected_Result, Actual_Result, Status

Priority Values: Critical, High, Medium, Low

Distribution: 60-70% of cases must be negative/validation tests (Critical or High priority).

Positive/success scenarios should be Low priority. Boundary tests should be Medium.

Task 2 — Automation Implementation

Create a complete automation solution covering all three modules.

Implement at least 20 automated test methods with assertions.

Prioritize critical and high-risk validations over happy-path scenarios.

Task 3 — Execution and Bug Detection

Execute automation. Provide execution evidence. Save all defects in a single consolidated file named `defect_report.json`. Detect and document at least 8 validation defects across all modules. Defects must be from actual test execution — not AI-predicted.

DELIVERABLES

1. Application Analysis & Requirements.txt — your documented analysis of all three modules including element IDs, acceptance criteria, and validation rules
2. Test case document — 60+ test cases covering all three modules in a csv file format
3. Automation codebase — page objects for Registration, Open New Account, Fund Transfer
4. `defect_report.json` — one consolidated defect report in JSON format capturing all defects from actual test execution

Test evaluation:

URL: <https://github.com/enterprises/CognizantGitHubCopilot>

Parabank Answer

MASTER PROMPT — PARA BANK UI AUTOMATION (SHORT & EFFECTIVE)

You are a **Senior QA Automation Engineer + SDET Expert**.

Your task is to complete a **UI Automation Assessment for ParaBank Application** covering:

- Registration Module (US-01 to US-07)
- Open New Account Module (US-08 to US-10)
- Fund Transfer Module (US-11 to US-13)

Use **industry best practices** in:

- Test design
- Automation framework design
- Validation handling
- Defect reporting

You **MUST** follow ALL instructions strictly.

◆ APPLICATION DETAILS

Registration URL:

<http://webapps.tekstac.com:9090/tekbank/register.htm>

Other modules are accessible after login.

◆ ELEMENT IDs (USE EXACTLY)

Registration

- customer.firstName
- customer.lastName

- customer.address.street
- customer.address.city
- customer.address.state
- customer.address.zipCode
- customer.phoneNumber
- customer.ssn
- customer.username
- customer.password
- repeatedPassword

Open New Account

- type
- fromAccountId

Fund Transfer

- amount
 - fromAccountId
 - toAccountId
-

◆ VALIDATION RULES (STRICT)

Registration

- Names + City → letters only
- State → exactly 2 letters
- ZIP → exactly 5 digits
- Phone → exactly 10 digits
- SSN → exactly 9 digits
- All 11 fields mandatory

Open Account

- Account Type → required
- Source Account → required (dynamic dropdown)
- Ignore "\$100.00" label

Fund Transfer

- Amount → required, numeric, > 0
- From Account → required
- To Account → required

◆ ACCEPTANCE CRITERIA (US-01 to US-13)

Use these EXACTLY while generating outputs:

- US-01 → Name validation
 - US-02 → Address & City validation
 - US-03 → State validation
 - US-04 → ZIP validation
 - US-05 → Phone & SSN validation
 - US-06 → Username/Password match
 - US-07 → Mandatory fields validation
 - US-08 → Account type selection
 - US-09 → Source account population
 - US-10 → Account creation success
 - US-11 → Transfer amount validation
 - US-12 → Account dropdown population
 - US-13 → Successful fund transfer
-

◆ STEP-BY-STEP EXECUTION (DO NOT SKIP)

Step 1 — Application Analysis

- Map all elements
- Identify field types
- Observe behavior for invalid/empty inputs

Step 2 — Acceptance Criteria

- Define for US-01 to US-13

Step 3 — Validation Rules

- Document all field-level validations

👉 Save Step 1–3 output in:
Application Analysis & Requirements.txt

Step 4 — Test Case Generation (CSV)

Generate **≥ 60 test cases** with:

Columns:

TC_ID, Module, User_Story_ID, Test_Scenario, Test_Type, Priority, Precondition,
Test_Steps, Test_Data, Expected_Result, Actual_Result, Status

Rules:

- 60–70% → Negative tests (Critical/High)
- Positive tests → Low
- Boundary tests → Medium
- Cover ALL 13 user stories

Step 5 — Automation Code

Generate a **complete automation framework** using ANY ONE:
(Selenium / Playwright / Cypress)

Include:

- Base setup
- Page Object Model (POM)
- Utility classes
- Test classes

Must implement:

- ≥ 20 automated test methods
- Assertions for validations
- Use given element IDs

Step 6 — Refinement

- Improve failing tests
- Fix locators/assertions
- Increase validation coverage

Step 7 — Defect Reporting

After execution:

Create **defect_report.json** with:

- At least 8 REAL defects
 - Each defect must include:
 - Module
 - Steps
 - Test Data
 - Expected Result
 - Actual Result
 - Severity
 - Priority
-

◆ **DELIVERABLES (MANDATORY)**

Application Analysis & Requirements.txt

- Must include:
 - Element IDs
 - Acceptance Criteria (US-01 to US-13)
 - Validation Rules

TestCases.csv

- Minimum 60 test cases
- Must include all required columns:
 - TC_ID, Module, User_Story_ID, Test_Scenario, Test_Type, Priority, Precondition, Test_Steps, Test_Data, Expected_Result, Actual_Result, Status

Automation Codebase

- Page Object Model (POM)
- Separate pages for:
 - Registration
 - Open New Account
 - Fund Transfer
- ≥ 20 test methods with assertions

defect_report.json

- Single consolidated file
- Minimum 8 defects
- Each defect must include:
 - Module
 - Steps

- Test Data
 - Expected Result
 - Actual Result
 - Severity
 - Priority
-

◆ IMPORTANT RULES

- DO NOT change workspace structure
 - Keep project clean (no redundant files)
 - Use VS Code terminal for execution
 - Follow naming conventions
 - Prioritize validation-heavy testing
-

◆ OUTPUT FORMAT REQUIREMENTS

- Test cases → CSV format
 - Code → structured files
 - Defects → JSON
 - Analysis → plain text
-

FINAL INSTRUCTION

Generate outputs **step-by-step (NOT all at once)**.
Wait for user confirmation before moving to next step.

If you want, I can also give you **step-wise mini prompts (Step 1 → Step 7)** like last time so you can *drive the AI perfectly and score higher*.